Knowledge Graphs & Graph Theory

A Comprehensive Mini-Book

From Foundations to Frontiers

Bernard

May 24, 2026

Contents

Preface

Knowledge graphs (KGs) have emerged as a cornerstone of modern artificial intelligence, bridging the gap between raw data and machine-interpretable knowledge. From Google's search graph to Wikidata's community-curated knowledge base, from GraphRAG-powered chatbots to ontology-constrained large language model reasoning, KGs are everywhere — yet their theoretical foundations and practical construction remain scattered across disparate fields: graph theory, database systems, natural language processing, semantic web, and machine learning.

This mini-book aims to provide a self-contained, rigorous treatment of knowledge graphs and their mathematical underpinnings. It is designed for graduate students, researchers, and practitioners who want a thorough understanding of the field — not just a surface-level tour.

What this book covers. We begin with graph theory (Chapter 1), establishing the mathematical language used throughout. Chapter 2 defines knowledge graphs proper, contrasting them with relational databases and vector stores. Chapters 3 and 4 cover data models (RDF, property graphs) and ontologies (RDFS, OWL). Chapter 5 walks through the end-to-end KG construction pipeline. Querying — SPARQL, Cypher, and the emerging GQL standard — is treated in Chapter 6.

The second half of the book turns to machine learning: knowledge graph embeddings (Chapter 7), graph neural networks (Chapter 8), and the integration of KGs with large language models via GraphRAG (Chapter 9). Chapter 10 focuses on knowledge graph question answering (KGQA), including constrained decoding approaches such as GCR and DCA-Trie. Production infrastructure and storage are covered in Chapter 11, and advanced topics — temporal KGs, hyper-relational facts, inductive reasoning, neuro-symbolic AI, quality assurance — in Chapter 12. We close with a structured learning roadmap (Chapter 13) and a comprehensive resource index (Chapter 14).

Prerequisites. Readers should be comfortable with basic linear algebra, probability, and set theory. Familiarity with Python is helpful for the code examples. No prior knowledge of graph theory or knowledge graphs is assumed.

Notation. We denote graphs as $G = (V, E)$, knowledge graphs as $\mathcal{K} = (\mathcal{E}, \mathcal{R}, T)$ where $\mathcal{E}$ is the entity set, $\mathcal{R}$ the relation set, and $T \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ the triple set. Vectors are bold lowercase ($\mathbf{h}$), matrices bold uppercase ($\mathbf{W}$), and sets in calligraphic ($\mathcal{E}$).

Bernard
May 24, 2026

Chapter 1

Graph Theory: The Foundation

Overview. Graph theory provides the mathematical language for describing and analysing networked structures. Every knowledge graph is, at its core, a graph. Understanding graph-theoretic concepts — from basic definitions to spectral theory — is essential for designing, querying, and learning from KGs. This chapter develops the foundations rigorously but accessibly, with an eye toward KG applications.

1.1 Basic Definitions

Definition 1.1 (Graph)**.** A *graph* is an ordered pair $G = (V, E)$ where V is a set of *vertices* (also called *nodes*) and E is a set of *edges*. Each edge is either:

- An unordered pair $\{u, v\}$ with $u, v \in V$ (undirected graph), or
- An ordered pair (u, v) with $u, v \in V$ (directed graph).

Definition 1.2 (Directed graph)**.** A *directed graph* (or *digraph*) $G = (V, A)$ consists of a vertex set V and an *arc* set $A \subseteq V \times V$. Arcs (u, v) have a direction from u (the *tail*) to v (the *head*).

Knowledge graphs are almost always directed graphs, since relationships have inherent direction: Elvis_Presley $\xrightarrow{\text{starred\_in}}$ Blue_Hawaii is not the same as the reverse.

Definition 1.3 (Labeled graph)**.** A *labeled graph* is a graph equipped with a labeling function $\ell : V \cup E \to \Sigma$ mapping vertices and/or edges to symbols in an alphabet Σ. In KGs, edges carry relation-type labels and vertices may carry type labels.

Definition 1.4 (Subgraph)**.** A graph $H = (V_H, E_H)$ is a *subgraph* of $G = (V, E)$, denoted $H \subseteq G$, if $V_H \subseteq V$ and $E_H \subseteq E$. An *induced subgraph* on $V' \subseteq V$ contains all edges of G whose endpoints are both in V'.

The induced subgraph is the natural notion of a "query context" in KG reasoning: given a set of entities of interest, we take all relations among them.

Definition 1.5 (Walk, path, cycle)**.** A *walk* of length k in a graph G is a sequence $v_0 e_1 v_1 e_2 \dots e_k v_k$ where each e_i connects v_{i-1} and v_i. A *path* is a walk with no repeated vertices. A *cycle* is a walk of length $k \geq 3$ that starts and ends at the same vertex with no other repeated vertices.

Definition 1.6 (Tree)**.** A *tree* is a connected, acyclic undirected graph. A *rooted tree* designates a special vertex as the root. Every tree with n vertices has exactly $n - 1$ edges. Trees are the natural structure for representing hierarchical knowledge (taxonomies, ontologies).

Definition 1.7 (Degree)**.** The *degree* $\deg(v)$ of a vertex v is the number of edges incident to it. In a directed graph, the *in-degree* $\deg^-(v)$ counts incoming arcs and *out-degree* $\deg^+(v)$ counts outgoing arcs.

KG relevance: The average out-degree of entities in a KG determines the branching factor for multi-hop traversal. Freebase has $\overline{\deg^+} \approx 20$, meaning a 3-hop BFS from a single entity can visit up to $20^3 = 8{,}000$ candidate paths.

Definition 1.8 (Graph isomorphism)**.** Two graphs $G = (V, E)$ and $G' = (V', E')$ are *isomorphic* if there exists a bijection $\phi : V \to V'$ such that $\{u, v\} \in E \iff \{\phi(u), \phi(v)\} \in E'$. The *graph isomorphism problem* asks whether such a bijection exists; its complexity class is not known to be in P nor NP-complete.

1.2 Graph Representations

Three standard computer representations for graphs, each with different trade-offs:

Table 1.1: Graph representation comparison

Representation	Space	Edge query	Best for
Adjacency matrix $A_{ij} = \mathbb{1}_{(i,j)\in E}$	$O(V^2)$	$O(1)$	Dense graphs, linear algebra
Adjacency list (per-vertex neighbor list)	$O(V + E)$	$O(\deg(v))$	Sparse graphs (most KGs)
Edge list (u, v) pairs	$O(E)$	$O(E)$	Simple processing, file formats

Definition 1.9 (Adjacency matrix)**.** For a graph $G = (V, E)$ with $V = \{1, \ldots, n\}$, the *adjacency matrix* $A \in \{0, 1\}^{n\times n}$ has entries $A_{ij} = 1$ if $(i, j) \in E$ and 0 otherwise. For undirected graphs, A is symmetric.

KGs require a third-order extension: the *adjacency tensor* $\mathcal{A} \in \{0, 1\}^{|\mathcal{E}|\times|\mathcal{R}|\times|\mathcal{E}|}$ where $\mathcal{A}_{ijk} = 1$ if triple (e_i, r_k, e_j) exists.

1.3 Graph Traversal

Algorithm 1 BFS for KG path enumeration

```
def bfs_paths(G, source, max_depth):
    """Enumerate all paths from source up to max_depth."""
    frontier = [(source, [source])]  # (entity, path_so_far)
    all_paths = []
    for depth in range(max_depth):
        next_frontier = []
        for entity, path in frontier:
            for relation, neighbor in G.out_edges(entity):
                if neighbor not in path:  # avoid cycles
                    new_path = path + [(relation, neighbor)]
                    next_frontier.append((neighbor, new_path))
                    all_paths.append(new_path)
        frontier = next_frontier
    return all_paths
```

This algorithm is the computational core of graph-constrained decoding methods (GCR, DCA-Trie). Its time complexity is $O(\overline{d}^k)$ where $\overline{d}$ is the average out-degree and k is the maximum path length.

1.4 Spectral Graph Theory

Definition 1.10 (Graph Laplacian)**.** For an undirected graph $G = (V, E)$ with adjacency matrix A and degree matrix $D = \mathrm{diag}(\deg(v_1), \ldots, \deg(v_n))$, the *graph Laplacian* is:

$$L = D - A$$

The *normalized Laplacian* is $\mathcal{L} = I - D^{-1/2} A D^{-1/2}$.

Theorem 1.1 (Properties of L)**.** *1. L is symmetric and positive semidefinite.*

2. *The smallest eigenvalue of L is 0, with eigenvector $\mathbb{1} = (1, \ldots, 1)^\top$.*
3. *The multiplicity of eigenvalue 0 equals the number of connected components of G.*
4. *For any vector $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{x}^\top L \mathbf{x} = \sum_{(i,j)\in E} (x_i - x_j)^2$.*

This last property makes the Laplacian central to graph signal processing and spectral clustering: it measures the "smoothness" of a function on the graph.

Proposition 1.1 (Spectral clustering)**.** *Given a graph G, the eigenvector corresponding to the second-smallest eigenvalue of L (the* Fiedler vector*) provides the optimal 2-way partition of V under the normalized cut criterion.*

Spectral methods are used in KG construction for entity resolution (clustering co-referent entities) and in GraphRAG for community detection.

1.5 The Weisfeiler–Lehman Hierarchy

The *Weisfeiler–Lehman (WL) test* is a polynomial-time algorithm for graph isomorphism testing that forms the theoretical foundation for graph neural network expressivity.

Definition 1.11 (1-WL color refinement)**.** Given a graph $G = (V, E)$ with initial colors $c^{(0)}(v)$, iterate:

$$c^{(t+1)}(v) = \mathrm{hash}\Big(c^{(t)}(v), \big\{\!\!\big\{ c^{(t)}(u) : u \in \mathcal{N}(v) \big\}\!\!\big\}\Big)$$

where the double braces denote a multiset. The algorithm halts when the color partition stabilizes.

Theorem 1.2 (GNN expressivity)**.** *Message-passing GNNs are at most as powerful as the 1-WL test for distinguishing non-isomorphic graphs. To exceed 1-WL, a GNN must use higher-order message passing (k-WL) or structural features (e.g., random walks, distance encoding).*

This result explains why simple GNNs fail on regular graphs and motivates architectures like GIN (Graph Isomorphism Network) that match 1-WL.

1.6 Exercises

Exercise 1.1. Prove that the sum of degrees in any undirected graph equals $2|E|$.

Exercise 1.2. Show that the graph Laplacian $L = D - A$ is positive semidefinite.

Exercise 1.3. Given a KG with average out-degree $\overline{d}$, derive the expected number of paths of length k reachable from a source entity. How does this grow with k?

Exercise 1.4. Implement the BFS path enumeration algorithm on a toy KG with 10 entities and 3 relation types. Count the number of paths at each depth.

Exercise 1.5. Prove that 1-WL cannot distinguish two non-isomorphic regular graphs of the same degree and size.

Chapter 2

What Is a Knowledge Graph?

Overview. Despite widespread use, "knowledge graph" resists a single precise definition. This chapter surveys existing definitions, establishes a working definition for this book, and contrasts KGs with related data management paradigms.

2.1 Toward a Definition

The term *knowledge graph* was popularised by Google in 2012, but the underlying ideas date back decades to semantic networks (Quillian, 1968), description logics, and early expert systems. A synthesis of existing definitions yields:

Definition 2.1 (Knowledge Graph)**.** A *knowledge graph* $\mathcal{K} = (\mathcal{E}, \mathcal{R}, T, \Sigma)$ consists of:

1. An entity set $\mathcal{E}$ (nodes).
2. A relation set $\mathcal{R}$ (edge types).
3. A triple set $T \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ (facts), where each triple $(h, r, t) \in T$ states that relation r holds between head h and tail t.
4. A schema Σ (optional) defining constraints on entities and relations — e.g., domain/range restrictions, class hierarchies, cardinality constraints.

Remark 2.1. This definition encompasses both *schema-free* KGs (Wikidata-like, where anyone can add any triple) and *schema-enforced* KGs (enterprise KGs with OWL ontologies). The difference matters for reasoning and quality assurance.

Example 2.1 (A KG triple)**.** The fact "Elvis Presley starred in the 1961 movie Blue Hawaii" decomposes into:

$$\begin{aligned} h &= \text{Elvis\_Presley} \in \mathcal{E}, \\ r &= \text{starred\_in} \in \mathcal{R}, \\ t &= \text{Blue\_Hawaii} \in \mathcal{E}. \end{aligned}$$

Additionally, a separate triple states the movie's release year:

$$(\text{Blue\_Hawaii}, \text{release\_year}, \text{“1961”})$$

where the object is a *literal* rather than an entity.

2.2 Ehrenberg's Four Characteristics

The seminal survey by Ehrenberg et al. (2024) distills four defining characteristics of KGs:

(1) Structured in graph form. Data is explicitly represented as nodes and edges, not as flattened tables or unstructured text.

(2) Normalized into triples. Each atomic fact is a subject-predicate-object triple, enabling fine-grained querying and reasoning.

(3) Connected via meaningful relationships. Edges carry semantic types (not just "link"), enabling multi-hop traversal over interpretable paths.

(4) Schematic and extensible. The schema can evolve — new relation types, entity types, and constraints can be added without rewriting existing data.

2.3 KG vs. Relational Database

A common question: why not just use a relational database? The answer lies in how each handles multi-hop queries and schema evolution.

Table 2.1: Knowledge graphs vs. relational databases

Dimension	Knowledge Graph	Relational Database
Relationships	First-class citizen, directly traversable	Foreign key joins
Multi-hop query	$O(d^k)$ native traversal	k JOINs, exponential optimizer cost
Schema	Flexible, can evolve on the fly	Rigid, requires ALTER TABLE migrations
Semantic reasoning	Supports inference (OWL, rules)	Exact matching only
Extensibility	Add new edge types without migration	New tables/columns needed
Completeness	Open-world: missing $\neq$ false	Closed-world: NULL = unknown

Example 2.2 (Multi-hop query)**.** *Query:* "Find all directors of movies featuring Elvis Presley."

In a KG, this is a 2-hop traversal:

$$\text{Elvis} \xrightarrow{\text{starred\_in}} \text{Movie} \xrightarrow{\text{directed\_by}} \text{Director}$$

In SQL, this requires two JOINs across at least three tables (casting, movies, directors). The query planner may choose a suboptimal join order, leading to exponential worst-case complexity.

2.4 KG vs. Vector Database

With the rise of embeddings and RAG, vector databases have become popular for semantic search. They are complementary to, not replacements for, KGs.

Remark 2.2. The strongest systems combine both: vector search for entity linking and initial retrieval, then KG traversal for structured multi-hop reasoning. This is the principle behind GraphRAG (Chapter 9).

2.5 Major Knowledge Graphs

Table 2.2: Knowledge graphs vs. vector databases

Query	Vector DB	Knowledge Graph
"Find companies funded by Sequoia"	Returns chunks *mentioning* Sequoia + funding	Returns exact FUNDED_BY edg
"Shared integrations between two products"	Multiple queries + manual intersection	Single Cypher/SPAR
Completeness guarantee	"Most relevant" chunks (may miss)	All matching edges (
Explainability	Black-box similarity score	Explicit traversal pat

Table 2.3: Major knowledge graphs compared

KG	Scale	Domain	Access
Wikidata	∼100M entities, 1.5B statements	General	SPARQL, dump
Freebase (deprecated)	∼50M entities	General	Migrated to Wikidata
DBpedia	∼40M entities	Wikipedia-derived	SPARQL, dump
YAGO	∼10M entities, 120M facts	General	Download
Google KG	∼7B entities	General	API (limited)
NELL	∼3M beliefs	Web-derived	Download
ConceptNet	∼8M nodes	Commonsense	API, download
WordNet	∼155K words	Linguistic	Download

Note: Scales are approximate as of 2025–2026. Wikidata grows continuously.

2.6 Exercises

Exercise 2.1. Consider the sentence "The CEO of OpenAI, Sam Altman, testified before Congress in 2023." Decompose it into KG triples. Identify which elements are entities and which are literals.

Exercise 2.2. Given a relational schema with tables: `Actors(id, name)`, `Movies(id, title, year)`, `Casting(actor_id, movie_id, role)`. Write SQL for the 2-hop query from Example 2.2. Count the number of JOINs required.

Exercise 2.3. Explain why KGs operate under the Open World Assumption and why this matters for query answering. Provide a concrete example where OWA yields different behaviour than CWA.

Chapter 3

Data Models: RDF vs. Labeled Property Graphs

Overview. Two dominant data models govern how knowledge graphs are stored and queried: the Resource Description Framework (RDF, a W3C standard) and the Labeled Property Graph (LPG, dominant in industry). Choosing between them depends on whether you prioritise interoperability and reasoning (RDF) or performance and edge properties (LPG). This chapter explains both in depth, contrasts their trade-offs, and introduces the emerging GQL standard.

3.1 RDF: The Resource Description Framework

RDF is the W3C-recommended framework for representing graph data on the web. Every fact is a *triple* $\langle s, p, o \rangle$ where the subject s and predicate p are Internationalized Resource Identifiers (IRIs), and the object o is either an IRI or a literal.

Definition 3.1 (RDF triple)**.** An *RDF triple* is a statement $\langle s, p, o \rangle$ where:

- s (subject) is an IRI or blank node.
- p (predicate) is an IRI.
- o (object) is an IRI, blank node, or literal.

The set of all triples forms an RDF graph.

Example 3.1 (RDF triple in Turtle syntax)**.**

```
@prefix ex: <http://example.org/> .
ex:Elvis_Presley ex:starred_in ex:Blue_Hawaii .
ex:Blue_Hawaii rdf:type ex:Movie .
ex:Elvis_Presley rdf:type ex:Person .
ex:Blue_Hawaii ex:release_year "1961"^^xsd:integer .
```

3.1.1 RDF Serialization Formats

Turtle (.ttl) Compact, human-readable; the most common format for authoring RDF.

RDF/XML The original W3C standard; verbose but widely supported.

JSON-LD JSON-based, popular in web APIs; supports context documents for compactness.

N-Triples (.nt) One triple per line, no abbreviations; ideal for streaming and large dumps.

TriG Extends Turtle to support multiple named graphs in one file.

3.1.2 RDF-star (RDF*)

Standard RDF cannot natively make statements *about* statements. RDF-star extends the data model with *reified triples as subjects*:

```
<< ex:Elvis_Presley ex:starred_in ex:Blue_Hawaii >>
    ex:accordingTo ex:IMDb ;
    ex:confidence "0.95"^^xsd:decimal .
```

This is critical for: provenance tracking (who said this fact?), confidence scoring (how sure are we?), temporal annotations (when was this true?), and source attribution.

Definition 3.2 (RDF-star)**.** An *RDF-star triple* allows a triple $\langle s', p', o' \rangle$ itself to appear as the subject or object of another triple. This enables concise reification without the standard quadruple-blowup pattern.

3.2 Labeled Property Graphs

The *Labeled Property Graph (LPG)* model, popularised by Neo4j, enriches nodes and edges with key-value properties and type labels.

Definition 3.3 (Property graph)**.** A *property graph* is a directed, labeled, attributed multigraph $G = (V, E, \ell, P)$ where:

- V is a set of vertices.
- $E \subseteq V \times V$ is a set of directed edges.
- $\ell : V \cup E \to 2^{\Sigma}$ assigns one or more labels to each vertex and edge.
- $P : (V \cup E) \times \mathcal{K} \to \mathcal{V}$ assigns property key-value pairs.

Example 3.2 (Property graph representation)**.** Node: `Elvis_Presley`

- Labels: Person, Musician
- Properties: {name: “Elvis Presley”, birth_date: “1935-01-08”}

Node: `Blue_Hawaii`

- Labels: Movie
- Properties: {title: “Blue Hawaii”, release_year: 1961}

Edge: `Elvis_Presley -[:STARRED_IN]-> Blue_Hawaii`

- Properties: {role: “Chad Gates”}

3.2.1 Key Advantages of LPG over RDF

1. **Properties on edges:** In RDF, adding a property to a relationship (e.g., “role in a film”) requires reification (creating a new node for the statement). In LPG, edge properties are native.
2. **No global IRI requirement:** Nodes can use local identifiers, making LPGs simpler for self-contained applications.
3. **Faster traversal:** Native graph storage optimises pointer-chasing, while RDF triple stores often use relational backends.

Table 3.1: RDF vs. LPG: comprehensive comparison

Criterion	RDF	LPG
Standardisation	W3C standard (2004, 2014)	De facto (Cypher), GQL standard (2024)
Properties on edges	Reification or RDF-star needed	Native
Schema/ontology	RDFS, OWL (built-in semantics)	Application-level
Reasoning	Built-in RDFS/OWL reasoning	Manual implementation
Query language	SPARQL (W3C standard)	Cypher, Gremlin, GQL
Interoperability	Excellent (Linked Data)	Limited
Multi-hop traversal	Triple store dependent	Fast (native graph)
Industry adoption	Academic, data integration	Enterprise, OLTP

3.3 Detailed Comparison

3.4 Query Languages Compared

Table 3.2: SPARQL vs. Cypher: query patterns side by side

Task	SPARQL (RDF)	Cypher (Neo4j)
Single-hop retrieve	`SELECT ?m WHERE { ex:E ex:starred_in ?m }`	`MATCH (:Person{n:'E'})-[:STARRED_IN]->(m) RETURN m`
Two-hop path	`SELECT ?x WHERE { ex:E ex:starred_in / ex:directed_by ?x }`	`MATCH (:Person{n:'E'})-[:STARRED_IN]->()-[:DI RETURN d`
Property filter	`FILTER(?year > 1960)`	`WHERE m.release_year > 1960`
Variable-length	`ex:E (ex:r1|ex:r2){1,3} ?x`	`MATCH (p)-[:R1|R2*1..3]->(x)`
Shortest path	Not natively supported	`shortestPath( (p1)-[*]-(p2) )`

3.5 GQL: The New ISO Standard

GQL (ISO 39075:2024) is the first international standard for property graph querying, unifying concepts from Cypher, PGQL, and SQL/XML. It supports:

- **Graph patterns:** ASCII-art style path matching (similar to Cypher).
- **Composable queries:** Subqueries, recursion, aggregation.
- **Property graph schema:** Labels, properties, and constraints.
- **Interoperability:** Standardised interaction between graph and relational systems.

Major vendors (Neo4j, Oracle, Amazon Neptune) are adopting GQL alongside existing query languages. GQL is expected to gradually become the lingua franca of property graph querying.

3.6 Exercises

Exercise 3.1. Take the triple "Barack Obama was born in Honolulu, Hawaii on August 4, 1961." Express it in (a) Turtle RDF, (b) JSON-LD, and (c) a property graph representation. Identify which aspects of the fact are easier to express in each representation.

Exercise 3.2. Explain why RDF-star was necessary. Construct an example of a provenance-aware triple and compare the RDF-star version with the standard reification version (which requires a blank node and 3–4 triples).

Exercise 3.3. Write a SPARQL query that retrieves all actors who starred in movies directed by directors born after 1950. Then write the equivalent Cypher query. Compare the syntax.

Chapter 4

Ontologies, Schemas, and Semantics

Overview. An ontology gives a knowledge graph its "knowledge" — it defines what kinds of things exist, how they relate, and what inferences are valid. Without an ontology, a KG is just a collection of triples. With one, it becomes a reasoning system capable of answering queries that go beyond explicitly stored facts.

4.1 What Is an Ontology?

Definition 4.1 (Ontology)**.** An *ontology* is a formal, explicit specification of a shared conceptualisation (Gruber, 1993). In practical terms, an ontology defines:

- **Classes:** Categories of entities (Person, Movie, Location).
- **Relations:** Types of relationships between classes (starred_in, directed_by).
- **Axioms:** Constraints and logical rules (disjointness, cardinality, transitivity).
- **Individuals:** Concrete instances of classes (Elvis_Presley is a Person).

Remark 4.1. The "shared" in Gruber's definition is crucial: an ontology represents a *consensus* view of a domain, enabling interoperability between systems that adopt it. This is why W3C standards for ontologies (RDFS, OWL) are designed for the open web.

Example 4.1 (Pizza ontology fragment)**.** A classic teaching example:

- Class: Pizza, subclass of Food.
- Class: Margherita, subclass of Pizza.
- Relation: hasTopping, domain Pizza, range Topping.
- Axiom: A Pizza must have at least one Topping.
- Axiom: Pizza and Dessert are disjoint (nothing can be both).

4.2 Ontology Languages

4.2.1 RDFS: RDF Schema

RDFS provides basic ontological primitives: class hierarchies, property hierarchies, domain and range constraints.

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix ex:   <http://example.org/> .

ex:Person rdf:type rdfs:Class .
ex:Movie  rdf:type rdfs:Class .
```

```
ex:starred_in rdf:type rdf:Property ;
              rdfs:domain ex:Person ;
              rdfs:range  ex:Movie .
ex:Musician rdfs:subClassOf ex:Person .
```

The key inference enabled by this schema: if (Elvis, starred_in, Blue_Hawaii) then we know:

- Elvis is a Person (from domain of starred_in).
- Blue_Hawaii is a Movie (from range of starred_in).

4.2.2 OWL 2: Web Ontology Language

OWL 2 provides much richer expressivity than RDFS, including:

Class axioms: Disjointness, equivalence, covering axioms.

Property axioms: Transitivity, symmetry, functional/inverse-functional, property chains.

Cardinality restrictions: "A Movie must have at least 1 director" (≥ 1).

Enumeration: A class defined by listing its members.

Qualified restrictions: "A Pizza must have at least 3 toppings, all Savoury".

Example 4.2 (OWL 2 in Turtle).

```
ex:Person rdf:type owl:Class .
ex:Movie  rdf:type owl:Class .
ex:Person owl:disjointWith ex:Movie .

ex:directed_by rdf:type owl:ObjectProperty ;
               rdfs:domain ex:Movie ;
               rdfs:range  ex:Person .

ex:hasParent rdf:type owl:ObjectProperty ;
             rdfs:domain ex:Person ;
             rdfs:range  ex:Person .

ex:hasAncestor rdf:type owl:TransitiveProperty .
ex:hasParent rdfs:subPropertyOf ex:hasAncestor .
```

From these axioms, an OWL reasoner can infer: if A hasParent B and B hasParent C, then A hasAncestor C.

4.2.3 SKOS: Simple Knowledge Organization System

SKOS is designed for taxonomies, thesauri, and subject heading systems — less expressive than OWL but simpler:

```
ex:MachineLearning rdf:type skos:Concept .
ex:DeepLearning    rdfs:subClassOf ex:MachineLearning .
ex:MachineLearning skos:prefLabel "Machine learning"@en ;
                   skos:altLabel "ML"@en ;
                   skos:definition "Field of AI..."@en .
```

4.2.4 SHACL: Shapes Constraint Language

SHACL validates RDF graphs against structural constraints — like a schema language but focused on validation rather than inference:

```
ex:PersonShape rdf:type sh:NodeShape ;
    sh:targetClass ex:Person ;
    sh:property [
        sh:path ex:birthDate ;
        sh:datatype xsd:date ;
        sh:minCount 1 ;
        sh:maxCount 1
    ] .
```

This declares: every Person must have exactly one birthDate, and its value must be an xsd:date. Violations are reported but do not change the data.

4.3 OWL 2 Profiles

OWL 2 is undecidable in full. Three profiles trade expressivity for computational tractability:

Table 4.1: OWL 2 profiles

Profile	Restrictions	Reasoning complexity	Use case
OWL 2 EL	No universal quantifiers, no cardinality	$O(n^3)$	Large biomedical ontologies (SNOM
OWL 2 QL	No property axioms, no existentials	$O(n)$ query answering	Database-backed KGs
OWL 2 RL	No disjunction, no existential to new individuals	Polynomial	Rule-based, scalable

4.4 Open World vs. Closed World

Definition 4.2 (Open World Assumption (OWA))**.** Under OWA, a statement that is not in the KG is considered *unknown*, not false. Formally: if $\mathcal{K} \not\models \phi$, we cannot conclude $\mathcal{K} \models \neg\phi$.

Definition 4.3 (Closed World Assumption (CWA))**.** Under CWA, a statement not in the database is considered *false*. Formally: if $\mathcal{K} \not\models \phi$, then $\neg\phi$ holds. This is the standard assumption in relational databases.

Example 4.3 (OWA vs. CWA)**.** Suppose the KG contains:

$$(\text{Elvis\_Presley}, \text{born\_in}, \text{Tupelo})$$

but does *not* contain any triple about Elvis's death date.

- Under OWA: "Elvis's death date is unknown" (he may or may not have one).
- Under CWA: "Elvis has no death date" (false — he died in 1977).

The OWA is philosophically appropriate for the open web, where absence of information does not imply non-existence. It is also more complex for query answering: a query like "Who has no birth date?" cannot be answered definitively under OWA.

4.5 Ontology Design Methodology

1. **Define competency questions.** "What questions must this ontology answer?" This frames the scope.
2. **Identify key concepts.** Extract nouns from competency questions.
3. **Identify relationships.** Extract verbs connecting the concepts.
4. **Define class hierarchy.** Use subclass/superclass relationships.
5. **Set constraints.** Domain, range, cardinality, disjointness.
6. **Add axioms.** Transitive, symmetric properties; property chains.
7. **Evaluate.** Test against competency questions using a reasoner.

Example 4.4 (Freebase ontology for DCA-Trie)**.** In the DCA-Trie project, the Freebase ontology provides:

- Entity types: Person, Location, Organization, Award, Film, Music, etc.
- Relations: `starred_in` has domain Person and range Movie.
- The domain/range declarations serve as "free" constraint signals: they are already in the KG and can prune invalid paths without training.

4.6 Reasoning and Inference

Definition 4.4 (Deductive reasoning in KGs)**.** Given a set of explicit triples T and an ontology Σ, deductive reasoning computes the closure T^* of all triples entailed by Σ:

$$T^* = \{\phi \mid \Sigma \cup T \models \phi\}$$

Example 4.5 (Transitive reasoning)**.** If the ontology declares `located_in` as transitive:

$$(\text{Memphis}, \text{located\_in}, \text{Tennessee})$$

$$(\text{Tennessee}, \text{located\_in}, \text{USA})$$

An OWL reasoner infers:

$$(\text{Memphis}, \text{located\_in}, \text{USA})$$

The two main reasoning strategies:

Forward chaining (materialisation): Precompute T^* at load time. Fast querying, expensive updates.

Backward chaining (query rewriting): Rewrite queries to include inferred patterns. Slower queries, cheap updates.

4.7 Exercises

Exercise 4.1. Design a small OWL ontology for a university domain with classes: Person, Student, Professor, Course, Department. Include disjointness axioms, domain/range constraints, and one transitive property.

Exercise 4.2. Given the RDFS axioms from Section 4.2, implement the RDFS entailment rules in Python. Specifically, implement the domain and range inference rules.

Exercise 4.3. Explain why OWA leads to non-monotonic behaviour in query answering. Provide a concrete SPARQL query that returns different results under OWA vs. CWA.

Chapter 5

Knowledge Graph Construction Pipeline

Overview. Building a knowledge graph from raw data is a multi-stage pipeline involving entity extraction, relation extraction, entity linking, fusion, and quality assurance. Modern approaches increasingly leverage LLMs for both extraction and resolution. This chapter covers the full pipeline end-to-end.

5.1 Pipeline Architecture

The canonical KG construction pipeline:

```
Raw Data --> Entity Extraction --> Relation Extraction
                                          |
                                          v
    Entity Resolution --> Knowledge Fusion --> Quality Assurance --> KG Store
```

Each stage presents distinct challenges and has seen substantial advances with neural methods.

5.2 Source Data Types

Table 5.1: Source data types and processing approaches

Source type	Approach	Tools	Difficulty
Structured (CSV, SQL)	Direct mapping, R2RML	Ontop, SPARQL CONSTRUCT	Low
Semi-structured (JSON, XML)	Path extraction, schema mapping	XSLT, JSON-LD framing	Medium
Unstructured (text)	NER + RE pipeline	spaCy, REBEL, LLMs	High

5.3 Named Entity Recognition (NER)

Definition 5.1 (NER)**.** Given a text D, identify all spans $\langle s_i, e_i, t_i \rangle$ where positions s_i to e_i in D mention an entity of type t_i.

Example 5.1. Input: "Elvis Presley was born in Tupelo, Mississippi." Output:

- `Elvis Presley` (Person) @ [0:13]
- `Tupelo` (Location) @ [27:33]
- `Mississippi` (Location) @ [35:46]

5.3.1 Approaches

Rule-based: Gazetteers, regular expressions. Fast, precise, low recall.

CRF-based: Linear-chain Conditional Random Field over token-level features. Good for well-formed text.

Neural BERT-based: Fine-tuned BERT + linear classifier (e.g., spaCy `en_core_web_trf`). State-of-the-art.

GLiNER: Compact NER model that handles arbitrary entity types specified at inference time. Ideal for domain-specific KGs.

LLM-based: Prompt GPT/LLaMA with instructions and a schema. Flexible but expensive.

5.4 Relation Extraction (RE)

Definition 5.2 (RE)**.** Given a text D and two entity mentions, determine the relationship (if any) between them.

5.4.1 Approaches

End-to-end (REBEL). REBEL (Babelscape) is a seq2seq model trained on 200 relation types. Input text, output triples directly:

```
from transformers import pipeline
triplet_extractor = pipeline("text2text-generation",
    model="Babelscape/rebel-large", tokenizer="Babelscape/rebel-large")

text = "Elvis Presley starred in Blue Hawaii."
triplets = triplet_extractor(text)
# Returns: [("Elvis Presley", "starred in", "Blue Hawaii")]
```

LLM-based extraction. Modern approaches use structured output schemas with LLMs:

```
from pydantic import BaseModel
from typing import List

class Triple(BaseModel):
    subject: str
    predicate: str
    obj: str

class ExtractionResult(BaseModel):
    triples: List[Triple]

# Using instructor/outlines for structured generation
# Prompt: "Extract all entity-relation triples from the following text..."
```

Key advantage: the LLM can use world knowledge to infer relations even when they are not explicitly stated in the text.

5.5 Entity Linking (Disambiguation)

Definition 5.3 (Entity linking)**.** Given a surface form mention m (e.g., “Paris”), identify the correct KG entity $e \in \mathcal{E}$ it refers to, or NIL if none.

The core challenge is *ambiguity*: "Paris" could refer to the French capital, Paris Hilton, or the mythological prince. Context disambiguates.

Table 5.2: Entity linking approaches

Method	Approach	Example tools
String similarity	Edit distance, n-gram overlap	Fuzzy string matching
Prior-based	Entity popularity (from Wikipedia links)	TAGME
Neural ED	BERT-based candidate ranking	GENRE, REL
LLM-based	Prompt LLM with context and candidate list	GPT-4, Claude

Definition 5.4 (GENRE)**.** GENRE (De Cao et al., 2021) formulates entity linking as autoregressive generation conditioned on the context, using a constrained trie over valid entity names to ensure output is always a valid KG entity.

5.6 Knowledge Fusion and Deduplication

Multiple sources may describe the same real-world entity differently. Fusion resolves these conflicts.

5.6.1 The Deduplication Pipeline

1. **Blocking:** Hash entities into buckets (by name prefix, year, type) — only compare within buckets.
2. **Similarity scoring:** Compare attribute similarity (name, description) and structural similarity (shared neighbours).
3. **Clustering:** Run connected components or hierarchical clustering over high-similarity pairs.
4. **Merge:** Create canonical entity, link aliases, resolve conflicting attribute values.

Example 5.2. Source A says ("IBM", "hq_location", "Armonk, NY"). Source B says ("International Business Machines", "headquarters", "Armonk, New York").

Fusion resolves: both refer to Wikidata Q37156. Merged attributes: {name: "IBM", aliases: {"International Business Machines"}, location: "Armonk, NY"}.

5.7 End-to-End LLM-Based Construction

Modern pipelines use LLMs for the entire extraction + linking pipeline:

1. **Crawl** and download source documents.
2. **Chunk** into segments of ∼2K–4K tokens.
3. **Extract:** For each chunk, prompt an LLM with a Pydantic schema to output triples with entity types.
4. **Resolve:** Pass extracted entities through an entity linker (GENRE, Wikidata API) to get KG IDs.
5. **Fuse:** Merge duplicate entities and triples.

6. **Validate:** Run SHACL validation, check consistency.
7. **Load:** Ingest into Neo4j or RDF store.

Tools: KnowledgeSDK, LangChain, Haystack, LlamaIndex all support this pipeline.

5.8 Quality Assurance

Table 5.3: KG quality dimensions

Dimension	Question	Method
Completeness	Are all facts present?	Rule mining, link prediction
Correctness	Are the facts true?	Human verification, cross-source
Consistency	Logical contradictions?	SHACL/OWL reasoning
Freshness	Up-to-date?	Temporal decay, refresh scheduling

5.9 Exercises

Exercise 5.1. Implement a minimal NER system using spaCy on a sample of Wikipedia text. Evaluate precision, recall, and F1 against gold-standard annotations.

Exercise 5.2. Build a KG construction pipeline for a small corpus (e.g., 5 Wikipedia articles): extract triples with an LLM, link entities to Wikidata, and load into a local RDF store using rdflib.

Exercise 5.3. Explain the blocking technique for entity resolution. Why is it necessary for large-scale deduplication? Implement a simple blocking scheme using the first letter of the entity name.

Chapter 6

Querying Knowledge Graphs

Overview. A knowledge graph is only as useful as our ability to query it. This chapter covers the two dominant query languages — SPARQL (for RDF) and Cypher (for property graphs) — along with query optimisation techniques and the emerging GQL standard.

6.1 SPARQL

SPARQL (SPARQL Protocol and RDF Query Language) is the W3C standard for querying RDF data. Its syntax resembles SQL, but operates on graph patterns instead of tables.

6.1.1 Basic Graph Pattern Matching

```
PREFIX ex: <http://example.org/>
SELECT ?movie ?year
WHERE {
  ex:Elvis_Presley ex:starred_in ?movie .
  ?movie ex:release_year ?year .
  FILTER(?year > 1960)
}
```

The WHERE clause defines a *basic graph pattern (BGP)* — a set of triple patterns that must match simultaneously. SPARQL evaluates the BGP by finding all variable bindings that satisfy all triple patterns.

6.1.2 SPARQL Query Forms

Table 6.1: SPARQL query forms

Form	Returns	Example
SELECT	Table of variable bindings	"Find all movies Elvis starred in"
CONSTRUCT	RDF graph	"Create co-star relationships"
ASK	Boolean	"Did Elvis work with Tom Jones?"
DESCRIBE	RDF graph	"Get all facts about Elvis"

6.1.3 Property Paths (Multi-hop)

SPARQL 1.1 introduced regular expression paths for multi-hop traversal:

```
# Two-hop: Elvis -> movie -> director
SELECT ?director WHERE {
  ex:Elvis_Presley ex:starred_in / ex:directed_by ?director
}
```

```
# Variable length: 1 to 3 hops
SELECT ?x WHERE {
  ex:Elvis_Presley (ex:starred_in|ex:produced_by){1,3} ?x
}

# Arbitrary length (Kleene star)
SELECT ?x WHERE {
  ex:Elvis_Presley ex:influenced* ?x
}
```

6.1.4 CONSTRUCT Queries

CONSTRUCT creates new RDF triples from query results, enabling inference at query time:

```
CONSTRUCT { ?s ex:worked_with ?o }
WHERE {
  ?s ex:starred_in ?m .
  ?o ex:starred_in ?m .
  FILTER(?s != ?o)
}
```

This materialises co-star relationships without storing them explicitly.

6.2 Cypher

Cypher is Neo4j's property graph query language, using ASCII-art pattern syntax:

```
// Find Elvis's movies
MATCH (p:Person {name: 'Elvis Presley'})-[:STARRED_IN]->(m:Movie)
RETURN m.title, m.release_year

// Two-hop with condition
MATCH (p:Person {name: 'Elvis Presley'})-[:STARRED_IN]->()
      -[:DIRECTED_BY]->(d:Person)
WHERE d.birth_year > 1930
RETURN d.name

// Variable-length path
MATCH path = (p:Person {name: 'Elvis Presley'})
             -[:STARRED_IN|PRODUCED_BY*1..3]->(x)
RETURN x.name, length(path) AS hops
```

6.2.1 Shortest Path

Cypher has native shortest-path support, which SPARQL lacks:

```
MATCH path = shortestPath(
  (p1:Person {name: 'Elvis Presley'})-[*]-
  (p2:Person {name: 'Tom Jones'})
)
RETURN path
```

6.3 Query Optimisation

6.3.1 General Principles

1. **Filter early:** Push WHERE conditions as deep as possible into the query plan.

2. **Index frequent patterns:** Entity labels, property equality, commonly joined relations.
3. **Limit intermediate results:** Use LIMIT for exploratory queries.
4. **Avoid Cartesian products:** Every disconnected MATCH/pattern multiplies the result set.

6.3.2 Join Ordering

The query optimiser's most critical decision is join order. For SPARQL:

$$P(x, y) \bowtie Q(y, z) \bowtie R(z, w)$$

Different orderings can differ in execution time by orders of magnitude. Modern optimisers use cardinality estimates from statistical summaries of the KG.

6.4 Exercises

Exercise 6.1. Given a KG with schema: Person, Movie, starred_in, directed_by, produced_by, release_year, birth_year. Write SPARQL queries for:

1. All movies directed by directors born after 1950 featuring actors who also starred in a Scorsese film.
2. The shortest path between two actors who have never co-starred.

Exercise 6.2. Write the equivalent Cypher queries. Which aspects are easier in each language?

Exercise 6.3. Explain how a SPARQL query optimizer would evaluate the query from (1a). What join orders are possible? Which is likely optimal and why?

Chapter 7

Knowledge Graph Embeddings

Overview. KGs are symbolic and discrete. Machine learning needs continuous representations. KG embeddings map entities and relations to vectors in $\mathbb{R}^d$ such that the KG's structural patterns are preserved as geometric relationships. This chapter develops the main families of embedding models, their training objectives, and their limitations.

7.1 The Embedding Problem

Definition 7.1 (KG embedding)**.** Given a knowledge graph $\mathcal{K} = (\mathcal{E}, \mathcal{R}, T)$, a *knowledge graph embedding* is a mapping:

$$\phi : \mathcal{E} \to \mathbb{R}^d, \quad \psi : \mathcal{R} \to \mathbb{R}^d$$

such that for each triple $(h, r, t) \in T$, a scoring function $f : \mathcal{E} \times \mathcal{R} \times \mathcal{E} \to \mathbb{R}$ satisfies:

$$f(\phi(h), \psi(r), \phi(t)) \gg f(\phi(h'), \psi(r), \phi(t'))$$

for randomly corrupted triples $(h', r, t') \notin T$.

The scoring function f is chosen to reflect a geometric intuition about how relations transform entity vectors.

7.2 Translational Models

7.2.1 TransE (Bordes et al., 2013)

The simplest and most influential embedding model. The intuition: for a true triple (h, r, t), the head entity embedding plus the relation embedding should approximate the tail entity embedding.

Definition 7.2 (TransE scoring)**.**

$$f(h, r, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_p$$

where $\|\cdot\|_p$ is the L_1 or L_2 norm. Higher scores indicate more plausible triples.

Example 7.1 (TransE geometric intuition)**.** If we learn embeddings such that:

$$\mathbf{h}_{\text{Elvis}} + \mathbf{r}_{\text{starred\_in}} \approx \mathbf{t}_{\text{Blue\_Hawaii}}$$
$$\mathbf{h}_{\text{Elvis}} + \mathbf{r}_{\text{starred\_in}} \approx \mathbf{t}_{\text{Jailhouse\_Rock}}$$

then $\mathbf{t}_{\text{Blue\_Hawaii}}$ and $\mathbf{t}_{\text{Jailhouse\_Rock}}$ should be close in vector space, reflecting that both are Elvis movies.

Proposition 7.1 (Limitations of TransE)**.** *TransE cannot model:*

- ***Symmetric relations:*** *If (h, r, t) and (t, r, h) both hold (e.g., `spouse_of`), TransE requires $\mathbf{r} = 0$ and $\mathbf{h} \approx \mathbf{t}$, collapsing the representation.*
- ***1-to-N relations:*** *If (h, r, t_1) and (h, r, t_2) both hold, TransE forces $\mathbf{t}_1 \approx \mathbf{t}_2$.*
- ***Composition:*** *Cannot represent $r_1 \circ r_2 = r_3$ (e.g., mother_of $\circ$ parent_of = grandmother_of).*

7.2.2 RotatE (Sun et al., 2019)

RotatE addresses TransE's limitations by embedding in complex space, with relations as rotations.

Definition 7.3 (RotatE scoring)**.** Embeddings are in $\mathbb{C}^d$ (i.e., d-dimensional complex vectors). For each triple:

$$\mathbf{t} = \mathbf{h} \circ \mathbf{r} \quad \text{(element-wise Hadamard product)}$$

where $|\mathbf{r}_i| = 1$ for all i (each component of $\mathbf{r}$ lies on the unit circle). The score is:

$$f(h, r, t) = -\|\mathbf{h} \circ \mathbf{r} - \mathbf{t}\|$$

Theorem 7.1 (RotatE expressivity)**.** *RotatE can model all four relational patterns simultaneously:*

- ***Symmetry:*** $\mathbf{r}_i = \pm 1$ *for all* i
- ***Antisymmetry:*** $\mathbf{r}_i \neq \pm 1$
- ***Inversion:*** $\mathbf{r}_2 = \overline{\mathbf{r}}_1$ *(complex conjugate)*
- ***Composition:*** $\mathbf{r}_3 = \mathbf{r}_1 \circ \mathbf{r}_2$

This makes RotatE strictly more expressive than TransE, DistMult, and Compléx.

7.3 Bilinear Models

7.3.1 DistMult (Yang et al., 2015)

Definition 7.4 (DistMult scoring)**.**

$$f(h, r, t) = \mathbf{h}^\top \operatorname{diag}(\mathbf{r})\, \mathbf{t} = \sum_{i=1}^{d} h_i \cdot r_i \cdot t_i$$

DistMult is simple and efficient ($O(d)$ per triple), but $f(h, r, t) = f(t, r, h)$, so it can only model symmetric relations.

7.3.2 ComplEx (Trouillon et al., 2016)

ComplEx extends DistMult to complex space, using the complex conjugate to break symmetry:

Definition 7.5 (ComplEx scoring)**.**

$$f(h, r, t) = \operatorname{Re}(\mathbf{h}^\top \operatorname{diag}(\mathbf{r})\, \overline{\mathbf{t}}) = \langle \operatorname{Re}(\mathbf{h}), \operatorname{Re}(\mathbf{r}), \operatorname{Re}(\mathbf{t}) \rangle + \langle \operatorname{Im}(\mathbf{h}), \operatorname{Re}(\mathbf{r}), \operatorname{Im}(\mathbf{t}) \rangle + \langle \operatorname{Re}(\mathbf{h}), \operatorname{Im}(\mathbf{r}), \operatorname{Im}(\mathbf{t}) \rangle - \langle \operatorname{Im}(\mathbf{h}), \mathbf{I}$$

The asymmetry comes from the conjugate operation: $f(h, r, t) \neq f(t, r, h)$ in general, enabling modelling of antisymmetric relations while retaining DistMult's efficiency.

7.4 Training KG Embeddings

7.4.1 Loss Functions

Given a set of true triples $\mathcal{D}^+$ and corrupted triples $\mathcal{D}^-$, the standard training objective is:

Definition 7.6 (Margin ranking loss)**.**

$$\mathcal{L} = \sum_{(h,r,t)\in\mathcal{D}^+} \sum_{(h',r,t')\in\mathcal{D}^-} \max(0, \gamma + f(h', r, t') - f(h, r, t))$$

where $\gamma > 0$ is a margin hyperparameter.

Alternative losses: binary cross-entropy (DistMult), negative log-likelihood (ComplEx), self-adversarial sampling (RotatE).

7.4.2 Negative Sampling

For each true triple, we corrupt either the head or tail:

```
def corrupt_triple(h, r, t, num_entities):
    if random.random() < 0.5:
        h_corrupt = random.randint(0, num_entities - 1)
        return (h_corrupt, r, t)
    else:
        t_corrupt = random.randint(0, num_entities - 1)
        return (h, r, t_corrupt)
```

Key insight: uniform negative sampling (choose random entity) is simple but produces easy negatives. Self-adversarial sampling (RotatE) weights negative samples by their current score, focusing on harder cases.

7.5 Evaluation Protocol

For link prediction (predicting missing $(h, r, ?)$ or $(?, r, t)$):

1. For each test triple (h, r, t), replace t with every entity $e \in \mathcal{E}$.
2. Score all $|\mathcal{E}|$ candidates.
3. Compute the rank of the true entity t.
4. Apply *filtered* setting: remove any corrupted triple that exists in the training/validation/test set.
5. Aggregate ranks into MRR and Hits@K.

Definition 7.7 (Evaluation metrics)**.**

$$\text{MRR} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{t\in\mathcal{D}_{\text{test}}} \frac{1}{\text{rank}(t)}$$

$$\text{Hits@K} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{t\in\mathcal{D}_{\text{test}}} \mathbb{1}_{\text{rank}(t)\le K}$$

Table 7.1: Model comparison on FB15k-237 (filtered)

Model	MRR	Hits@1	Hits@10
TransE	0.294	–	0.465
DistMult	0.241	0.155	0.419
ComplEx	0.278	0.194	0.450
RotatE	0.338	0.241	0.533
TuckER	0.358	0.266	0.544
NBFNet (GNN)	0.417	0.321	0.582

7.6 Exercises

Exercise 7.1. Show algebraically why TransE cannot model symmetric relations. Prove that if (h, r, t) and (t, r, h) are both true, TransE forces $\mathbf{r} = 0$ and $\mathbf{h} \approx \mathbf{t}$.

Exercise 7.2. Implement the scoring function for DistMult and RotatE in PyTorch. Compare the number of parameters each model requires for $|\mathcal{E}| = 10{,}000$, $|\mathcal{R}| = 100$, $d = 200$.

Exercise 7.3. Explain the "filtered setting" for KG embedding evaluation. Why is it necessary? What happens if we do not filter out corrupted triples that exist in the training set?

Chapter 8

Graph Neural Networks for KGs

Overview. Graph Neural Networks (GNNs) learn node representations via iterative message passing between neighbours. For KGs, GNNs offer context-dependent representations (the same entity gets different embeddings for different query relations) and inductive capabilities (handling unseen entities). This chapter covers the main GNN architectures, their application to KGs, and the theoretical limits of their expressivity.

8.1 The Message-Passing Paradigm

Definition 8.1 (Message-passing GNN)**.** Let $G = (V, E)$ be a graph with node features $\mathbf{x}_v \in \mathbb{R}^{d_0}$. A message-passing GNN computes representations through L layers:

$$\mathbf{m}_v^{(l)} = \mathrm{AGG}^{(l)}\Big(\Big\{\phi^{(l)}(\mathbf{h}_u^{(l-1)}, \mathbf{h}_v^{(l-1)}, \mathbf{e}_{uv}) : u \in \mathcal{N}(v)\Big\}\Big)$$
$$\mathbf{h}_v^{(l)} = \mathrm{UPDATE}^{(l)}(\mathbf{h}_v^{(l-1)}, \mathbf{m}_v^{(l)})$$

where $\mathbf{h}_v^{(0)} = \mathbf{x}_v$, $\mathcal{N}(v)$ is the neighbourhood of v, and $\mathbf{e}_{uv}$ are optional edge features.

Analogy: A GNN is a dinner party. Each person (node) talks to their neighbours, aggregates what they hear (AGG), and updates their own understanding (UPDATE). After L rounds, everyone has a view of the conversation within their L-hop radius.

8.2 Key Architectures

8.2.1 Graph Convolutional Network (GCN)

GCN (Kipf & Welling, 2017) uses mean-pooling with symmetric normalisation:

Definition 8.2 (GCN layer)**.**

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}^{(l)} \sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\deg(u)\deg(v)}} \mathbf{h}_u^{(l-1)}\right)$$

The normalisation factor $\frac{1}{\sqrt{\deg(u)\deg(v)}}$ prevents degree-biased representations and stabilises training.

8.2.2 Graph Attention Network (GAT)

GAT (Veličković et al., 2018) learns attention weights for each neighbour:

Definition 8.3 (GAT layer)**.**

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

where attention coefficients are:

$$\alpha_{vu} = \frac{\exp(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \| \mathbf{W}\mathbf{h}_u])}{\sum_{w \in \mathcal{N}(v)} \exp(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \| \mathbf{W}\mathbf{h}_w])}$$

and $\|$ denotes concatenation.

Multi-head attention (concatenating/averaging K independent attention heads) improves stability.

8.2.3 GraphSAGE (Hamilton et al., 2017)

GraphSAGE introduced *inductive* learning — generalising to unseen nodes at inference:

Definition 8.4 (GraphSAGE layer)**.**

$$\mathbf{h}_v^{(l)} = \sigma\Big(\mathbf{W}^{(l)} \cdot \Big[\mathbf{h}_v^{(l-1)} \Big\| \mathrm{AGG}^{(l)}\Big(\Big\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\Big\}\Big)\Big]\Big)$$

where AGG can be mean, LSTM, or max-pooling over neighbours.

8.2.4 Relational Graph Convolution (R-GCN)

For KGs with multiple relation types, R-GCN uses relation-specific transformations:

Definition 8.5 (R-GCN layer)**.**

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}_0^{(l)}\mathbf{h}_v^{(l-1)} + \sum_{r \in \mathcal{R}} \sum_{u \in \mathcal{N}_r(v)} \frac{1}{c_{v,r}} \mathbf{W}_r^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

where $\mathcal{N}_r(v)$ are neighbours connected by relation r, and $c_{v,r}$ is a normalisation constant.

Remark 8.1. R-GCN's per-relation weight matrices $\mathbf{W}_r$ lead to $O(|\mathcal{R}| \cdot d^2)$ parameters. For KGs with thousands of relations, this becomes prohibitive. Basis decomposition and block-diagonal matrices reduce the parameter count.

8.3 GNNs for Link Prediction

8.3.1 NBFNet (Zhu et al., 2021)

NBFNet formulates link prediction as label propagation: given a query $(h, r, ?)$, run a GNN with the head entity as the source of a "query signal".

Key insight: unlike TransE (which stores a single vector per entity), NBFNet computes representations *contextually* — the same entity gets different representations for different query relations.

8.4 Expressivity Limits

Theorem 8.1 (1-WL equivalence)**.** *Message-passing GNNs are at most as powerful as the 1-WL graph isomorphism test (Xu et al., 2019; Morris et al., 2019). Specifically:*

- *GCN, GAT, GraphSAGE are strictly less powerful than 1-WL.*
- *GIN (with sum aggregation and MLP update) matches 1-WL.*
- *No MP-GNN can distinguish regular graphs of the same degree.*

Corollary 8.1. *For link prediction in KGs, MP-GNNs cannot distinguish non-isomorphic neighbourhood structures that share the same multiset of node features. This motivates structural features (positional encodings, random walk features) as additional inputs.*

Algorithm 2 NBFNet forward pass for link prediction

```
def nbfnet_link_predict(kg, h, r, num_layers):
    # Initialise: h gets score 1, all others 0
    scores = {e: 0 for e in kg.entities}
    scores[h] = 1.0

    for layer in range(num_layers):
        new_scores = {}
        for e in kg.entities:
            # Message from incoming neighbours
            msg = 0
            for (rel, source) in kg.in_edges(e):
                msg += W_rel * scores[source]
            new_scores[e] = UPDATE(scores[e], msg)
        scores = new_scores

    return scores
```

8.5 Exercises

Exercise 8.1. Implement a single GCN layer in PyTorch. Verify that the normalisation factor $\frac{1}{\sqrt{\deg(u)\deg(v)}}$ prevents representation collapse for high-degree nodes.

Exercise 8.2. Compare the parameter count of GCN, GAT (with K heads), and R-GCN for $d = 256$, $|\mathcal{R}| = 50$, $K = 4$.

Exercise 8.3. Explain why GIN's sum aggregation is more powerful than mean or max aggregation. Provide a counterexample where mean aggregation fails to distinguish two non-isomorphic graphs.

Chapter 9

Knowledge Graphs and LLMs: GraphRAG

Overview. Large Language Models (LLMs) have revolutionised NLP but suffer from hallucinations, staleness, and lack of structured reasoning. Knowledge graphs provide precise, structured, grounded facts. GraphRAG — the integration of KGs with LLM-based retrieval-augmented generation — combines the strengths of both paradigms. This chapter surveys the GraphRAG landscape, from basic retrieval to constrained decoding.

9.1 The Limits of Standard RAG

Standard Retrieval-Augmented Generation (RAG) retrieves text chunks via vector similarity:

```
Query --> Embed --> Vector Search --> Top-k Chunks --> LLM --> Answer
```

Limitations:

1. **No multi-hop:** Cannot answer "What companies use technology X that competitor Y also uses?" — requires joining information across chunks.
2. **No completeness guarantee:** Returns "most relevant" chunks, may miss critical information.
3. **No structured reasoning:** Cannot traverse relationships.
4. **Atomicity:** A single chunk may contain multiple facts, making attribution difficult.

9.2 GraphRAG Architecture

The GraphRAG pipeline augments (or replaces) vector retrieval with KG traversal:

```
Query
  |
  v
Entity Linking (text --> KG entities)
  |
  v
Graph Traversal (BFS/DFS over KG paths)
  |
  v
Subgraph Extraction (paths or neighbourhood)
  |
  v
LLM Prompting (question + subgraph context)
  |
  v
Answer
```

Example 9.1 (Multi-hop GraphRAG)**.** Q: "Who is the CEO of the company that acquired GitHub?"

1. Entity link: "GitHub" → `github_entity`
2. Traverse `ACQUIRED_BY`: `github_entity` → `Microsoft`
3. Traverse `CEO_IS`: `Microsoft` → `Satya_Nadella`
4. Prompt: "Based on the KG: [GitHub] –acquired_by– [Microsoft] –ceo_is– [Satya Nadella]. Answer: Satya Nadella"

9.3 GraphRAG Systems

Table 9.1: GraphRAG systems compared

System	Key idea	KG construction	Year
Microsoft GraphRAG	Community detection + LLM summarisation	LLM-based	2024
LightRAG	Dual-level (entity + relation) retrieval	LLM-based	2024
HippoRAG	PPR over KG + vector store hybrid	NLP pipeline	2024
HippoRAG 2	Improved LLM-based KG construction	LLM-based	2025
GNN-RAG	GNN relevance scoring + path retrieval	From KGQA dataset	2025
GFM-RAG	Graph Foundation Model for RAG	Pretrained	2025

9.3.1 Microsoft GraphRAG in Detail

1. **Extract:** LLM extracts entities + relationships from each document chunk.
2. **Detect communities:** Leiden algorithm finds hierarchical communities in the extracted KG.
3. **Summarise:** LLM generates a summary for each community at each hierarchy level.
4. **Query:**
   - *Local search:* Find relevant entities, traverse neighbours, retrieve community summaries.
   - *Global search:* Retrieve high-level community summaries that cover the entire corpus.
5. **Answer:** LLM synthesises retrieved information into a final answer.

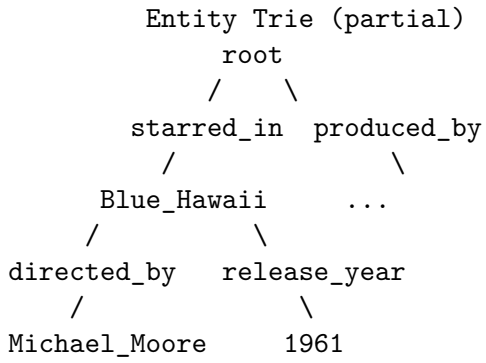
Key advance over standard RAG: answers requiring global understanding ("What are the main themes of this corpus?").

9.4 KG-Augmented Generation: Beyond Retrieval

GraphRAG retrieves KG facts as context. A more powerful integration *constrains* the LLM's generation to follow KG-valid paths.

9.4.1 Graph-Constrained Reasoning (GCR)

Definition 9.1 (GCR (Luo et al., 2025))**.** GCR builds a static trie over all KG paths reachable from a source entity. During generation, it masks logits so the LLM can only produce tokens forming valid KG paths. This guarantees 100% structural faithfulness.



9.4.2 Dynamic Oracle-Guided Decoding (DoG)

DoG (Li et al., 2025) addresses GCR's limitation: GCR must enumerate all paths up front (exponential). DoG expands the trie *dynamically*, adding child entities only after the parent is committed.

9.4.3 DCA-Trie: Ontology-Aware Constrained Decoding

DCA-Trie extends GCR with *type oracle gates*: before expanding a relation from an entity, check whether the relation's range matches the expected type (from the KG ontology). This prunes semantically irrelevant paths without reducing path recall.

9.5 Entity Linking for GraphRAG

Table 9.2: Entity linking tools for GraphRAG

Tool	Approach	Licence
REL (Radford et al.)	Neural ED with candidate generation	MIT
GENRE (De Cao et al.)	Autoregressive entity retrieval	MIT
Wikidata API	Direct search	Free
BLINK	BERT-based biencoder + cross-encoder	MIT
LLM-based	Prompt-based identification	N/A

9.6 Exercises

Exercise 9.1. Implement a minimal GraphRAG pipeline: given a query, link entities to Wikidata (via API), traverse 2 hops, and prompt an LLM with the retrieved subgraph.

Exercise 9.2. Compare standard RAG vs. GraphRAG on a multi-hop question (e.g., "What is the capital of the country where the Eiffel Tower is located?"). Explain why standard RAG fails.

Exercise 9.3. Explain the GCR trie construction algorithm. What is its time complexity? How does DoG reduce this complexity? Under what conditions does DCA-Trie reduce the trie size without losing valid paths?

Chapter 10

Knowledge Graph Question Answering

Overview. Knowledge Graph Question Answering (KGQA) is the task of answering natural language questions using facts stored in a KG. It is the primary downstream application driving KG research. This chapter surveys the three main paradigms — semantic parsing, retrieval-based reasoning, and constrained decoding — and their evaluation.

10.1 Problem Definition

Definition 10.1 (KGQA)**.** Given a natural language question q and a knowledge graph $\mathcal{K} = (\mathcal{E}, \mathcal{R}, T)$, find the answer entity (or entities) $A \subseteq \mathcal{E}$ such that:

1. There exists a reasoning path $e_0 \xrightarrow{r_1} e_1 \xrightarrow{r_2} \cdots \xrightarrow{r_k} e_k$ in $\mathcal{K}$ where e_0 is the topic entity and $e_k \in A$.
2. The path corresponds to the semantics of q.

Example 10.1. Q: "What award did Elvis Presley win in 1971?" Reasoning path:

$$\text{Elvis\_Presley} \xrightarrow{\text{award\_won}} \text{Grammy\_Award} \xrightarrow{\text{year}} 1971$$

Answer: Grammy Award

10.2 Benchmarks

Table 10.1: KGQA benchmarks

Dataset	Questions	Max hops	Source KG	Split
WebQSP	4,737	2	Freebase	train/test
ComplexWebQuestions	34,689	4	Freebase	train/dev/test
WebQuestions	5,810	2	Freebase	train/test
LC-QuAD 2.0	24,907	Multi	Wikidata	train/test
GrailQA	64,331	4	Freebase	train/dev/test

10.3 Semantic Parsing

Translate question → executable logical form → run against KG.

```
Q: "Who directed Titanic?"
  --> SPARQL: SELECT ?d WHERE { :Titanic :directed_by ?d }
  --> Execute --> Answer: James Cameron
```

10.3.1 Approaches

Grammar-based: Define a formal grammar for the target logical form. Zettlemoyer and Collins (2005) use CCG (Combinatory Categorial Grammar).

Seq2seq: Encode question with BERT/LSTM, decode logical form with attention. Requires large annotated datasets.

LLM-based: Prompt an LLM to generate SPARQL/Cypher directly. Works well for simple queries but degrades for complex ones.

Example 10.2 (LLM-based semantic parsing)**.**

```
Prompt: "Generate a SPARQL query to find all actors
         who starred in movies directed by Steven Spielberg."

Expected output:
PREFIX ex: <http://example.org/>
SELECT ?actor WHERE {
  ?actor ex:starred_in ?movie .
  ?movie ex:directed_by ex:Steven_Spielberg .
}
```

Remark 10.1. LLM-based semantic parsing is the most practical approach for production, but hallucination (generating non-existent relations/entities) remains a challenge. Constrained decoding (Section 10.5) addresses this.

10.4 Retrieval-Based KGQA

Instead of parsing to an executable query, retrieve a relevant subgraph and reason over it.

10.4.1 QA-GNN (Yasunaga et al., 2021)

Jointly encodes the question and KG subgraph:

1. Identify topic entities in the question.
2. Extract a k-hop subgraph around topic entities.
3. Run a GNN over the subgraph, with the question embedding as a global node.
4. Classify answer nodes.

10.4.2 GNN-RAG (Mavromatis & Karypis, 2025)

Two-stage approach:

1. **Relevance scoring:** GNN scores each KG entity's relevance to the question.
2. **Path retrieval:** Retrieve top-k shortest paths between topic entities and high-relevance entities.
3. **LLM reasoning:** Feed retrieved paths as context to an LLM for answer generation.

GNN-RAG achieves state-of-the-art on WebQSP (82.4% Hits@1) and CWQ (67.1%).

10.5 Constrained Decoding for KGQA

The most recent paradigm: rather than retrieve-and-reason (retrieval-based) or translate-to-query (semantic parsing), *constrain* the LLM's token generation to KG-valid paths.

Definition 10.2 (Constrained decoding for KGQA)**.** Given a question q and topic entity e_0, constrain the LLM to generate a sequence $\langle r_1, e_1, r_2, e_2, \ldots, e_k \rangle$ such that each (e_{i-1}, r_i, e_i) is a valid KG triple. This is achieved by masking the softmax over the vocabulary to allow only tokens that correspond to valid next elements.

10.5.1 Graph-Constrained Reasoning (GCR)

GCR (Luo et al., 2025, ICML) builds a static trie over all paths from e_0 up to depth k:

1. BFS from e_0 up to k hops, collecting all paths.
2. Build a trie from the collected paths.
3. During generation, mask logits so only trie-valid completions are possible.
4. Decode with beam search over trie paths.

Guarantee: 100% structural faithfulness — every generated path exists in the KG.
Limitation: The trie grows as $O(\overline{d}^k)$ where $\overline{d}$ is average degree.

10.5.2 Dynamic Oracle-Guided Decoding (DoG)

DoG (Li et al., 2025, ACL) expands the trie step by step:

1. At step i, given entity e_{i-1}, query the KG for all outgoing (r, e) pairs.
2. Add them to the current trie level.
3. Mask logits to allow only the retrieved relations/entities.

Advantage: No upfront path enumeration. Memory grows as $O(\overline{d})$ per step rather than $O(\overline{d}^k)$.

10.5.3 DCA-Trie: Ontology-Aware Pruning

The **Domain-Constraint-Aware Trie (DCA-Trie)** extends GCR with three oracle gates:

1. **Type oracle:** For each candidate relation r, check if the current entity's type matches r's domain.
2. **Range oracle:** Use r's range to prefer entities of compatible type.
3. **Domain oracle:** Filter paths by the expected answer type.

Effect: DCA-Trie reduces the constraint set by 30–45% compared to GCR on Freebase queries, while maintaining perfect path recall (100%).

Table 10.2: Constrained decoding methods comparison

Method	Trie construction	Memory	Faithfulness	Pruning
GCR	Static (all paths upfront)	$O(\overline{d}^k)$	100%	None
DoG	Dynamic (stepwise)	$O(\overline{d})$	100%	None
DCA-Trie	Static + oracle pruning	$O(\overline{d}^k)$ reduced	100%	30–45%

10.6 Evaluation Metrics

Hits@1: Fraction of questions where the top-1 predicted answer matches ground truth.

F1: Token-level overlap between predicted and ground-truth answer strings.

Path accuracy: For constrained decoding, fraction of generated paths that are KG-valid (should be 100% for GCR/DoG/DCA-Trie).

SIR: Semantic Irrelevance Ratio — fraction of valid-but-irrelevant paths remaining in the constraint set after pruning.

10.7 Exercises

Exercise 10.1. Given topic entity e_0 with out-degree $\deg^+(e_0) = 15$, average KG degree $\overline{d} = 20$, and $k = 3$, compute the expected trie size for GCR vs. DoG. Assume DCA-Trie prunes 40% of branches.

Exercise 10.2. Implement a minimal constrained decoding loop for a single KG path: given an entity e_0, query the KG for outgoing edges, mask the LLM vocabulary to allow only valid next relations, and decode one step.

Exercise 10.3. Explain why 100% path faithfulness does not guarantee 100% answer accuracy. What are the sources of error in a constrained decoding KGQA system?

Chapter 11

Storage and Production Infrastructure

Overview. Behind every production KG is a graph database system that handles storage, indexing, querying, and scaling. This chapter surveys the major graph databases, their trade-offs, scaling strategies, and guidance on when (and when not) to use a graph database.

11.1 Graph Databases

Table 11.1: Major graph databases

Database	Model	Query language	Best for
Neo4j	LPG	Cypher, GQL	OLTP, most popular, ACID
Amazon Neptune	LPG + RDF	Gremlin, SPARQL	AWS-native, multi-model
TigerGraph	LPG	GSQL	Large-scale analytics
ArangoDB	Multi-model	AQL	Mixed graph + document
Virtuoso	RDF	SPARQL	Large RDF stores
Stardog	RDF	SPARQL, OWL	Enterprise reasoning
GraphDB	RDF	SPARQL, SHACL	Semantic web
Jena Fuseki	RDF	SPARQL	Lightweight research
Dgraph	LPG (custom)	DQL	Distributed, low latency

11.2 Storage Internals

11.2.1 Native Graph Storage

Neo4j uses *index-free adjacency*: each node stores direct pointers (file offsets) to its relationships. Traversing from one node to its neighbour requires no index lookup — just a pointer dereference.

$$O(1) \text{ per edge traversal}$$

This is why native graph DBs outperform relational DBs on multi-hop queries by orders of magnitude.

11.2.2 Indexing Strategies

Token indices: Fast lookup of nodes by label/type.

Property indices: B-tree or hash index on frequently-queried properties.

Composite indices: Multi-property index for combined filters.

Full-text search: Lucene-based (Neo4j) for text properties.

Spatial indices: Geospatial queries on location data.

11.3 Scaling

11.3.1 Partitioning

Sharding a graph is fundamentally harder than sharding relational tables because edges cross machine boundaries.

Definition 11.1 (Edge-cut partitioning)**.** Assign vertices to machines $M_1, \ldots, M_p$. Edges whose endpoints are on different machines become *cross-partition*, requiring remote calls during traversal.

Definition 11.2 (Vertex-cut partitioning)**.** Assign each edge to a machine, replicating vertices across partitions. Reduces cross-partition traversal at the cost of increased storage.

Tools: METIS, ParMETIS, and PowerGraph (vertex-cut) for partitioning large graphs.

11.3.2 Distributed Graph Databases

Amazon Neptune: Managed, clustered (up to 128 TB), read replicas, multi-AZ.

TigerGraph: MPP (massively parallel processing), up to hundreds of nodes, automatic sharding.

JanusGraph: Open-source, uses HBase/Cassandra for storage, Elasticsearch for indexing.

Dgraph: Distributed by design, uses Raft consensus, low-latency.

11.4 Caching Strategies

Neighbourhood cache: Pre-fetch 2-hop neighbourhoods for frequently queried entities.

Pattern cache: Cache results of common query templates (e.g., "top-10 movies by genre" with parameterised genre).

Embedding cache: Cache precomputed entity embeddings for ML workloads, updating on a TTL schedule.

Result cache: Short-lived cache for identical query strings.

11.5 When Not to Use a Graph Database

Graph databases are not a panacea. Avoid them when:

1. **Pure aggregation:** SUM, COUNT, GROUP BY are faster in columnar/relational stores.

2. **Trivial relationships:** Flat data with a single join type (e.g., key-value with a secondary index).
3. **Full-text search:** Elasticsearch or vector DB for semantic search over text.
4. **High-volume simple lookups:** Key-value stores (Redis, DynamoDB) for single-key lookups.
5. **Analytics-only:** Data warehouse (Snowflake, Redshift) for analytical queries over billions of rows.

The sweet spot for graph databases is *multi-hop traversal over richly connected data* — exactly the KGQA use case.

11.6 Exercises

Exercise 11.1. Compare the storage representation of a 3-hop path $(e_0, r_1, e_1, r_2, e_2, r_3, e_3)$ in a relational database (triples table) vs. a native graph database (pointer-based). Estimate the I/O cost for each.

Exercise 11.2. Explain why edge-cut partitioning is NP-hard. How do METIS and ParMETIS approximate the optimal partition?

Exercise 11.3. Given a KG with 100M entities and 1B triples, estimate the storage requirements for Neo4j (native) vs. a relational triple store. What factors dominate the storage cost?

Chapter 12

Advanced Topics

Overview. This chapter surveys frontier research topics in knowledge graphs: temporal and hyper-relational KGs, inductive reasoning, neuro-symbolic AI, and quality assurance.

12.1 Temporal Knowledge Graphs

Definition 12.1 (Temporal triple)**.** A *temporal triple* extends a standard triple with a time interval:

$$(h, r, t, [\tau_{\text{start}}, \tau_{\text{end}}])$$

where τ_{start} and τ_{end} are timestamps indicating when the fact was valid.

Example 12.1.

$$(\text{Elvis\_Presley}, \text{married\_to}, \text{Priscilla\_Presley}, [1967, 1973])$$

$$(\text{Elvis\_Presley}, \text{married\_to}, \text{Priscilla\_Presley}, [1967, \infty)) \text{ is INFERRED false}$$

12.1.1 Temporal KG Embeddings

TComplEx (Lacroix et al., 2020): Extends Compléx with time-aware scoring. Each timestamp has an embedding that modulates entity representations.

TeRo (Xu et al., 2020): Relations as time-aware rotations in complex space.

T-GAP (Jung et al., 2021): Temporal graph attention network for event forecasting.

ChronoR: Rotations in quaternion space, capturing both temporal and relational structure.

12.1.2 Key Tasks

1. **Temporal link prediction:** Predict $(h, r, ?, \tau)$ given known facts up to time τ.
2. **Event forecasting:** Predict what facts will become true at future time $\tau > \tau_{\max}$.
3. **Temporal reasoning:** Answer queries involving temporal constraints ("Which award did Elvis win after 1970?").

12.2 Hyper-Relational Knowledge Graphs

Wikidata uses *qualifiers* — additional key-value pairs that contextualise a statement:

Example 12.2 (Wikidata qualifier pattern)**.**

```
wd:Elvis_Presley  wdt:award_received  wd:Grammy_Award .
                  pq:point_in_time    "1975"^^xsd:year ;
                  pq:category         "Best Pop Vocal" .
```

Standard models treat this as a set of independent triples, losing the connection between the award and its category/year. Hyper-relational models preserve this structure.

StarE (Galkin et al., 2020): Encodes the primary triple and all qualifiers jointly using an attention mechanism.

QUAD (Tran et al., 2023): Treats each hyper-relational fact as a 4th-order tensor.

HINGE: Inductive hyper-relational reasoning with GNNs.

12.3 Inductive KG Reasoning

Inductive methods generalise to entities unseen during training. This is critical for production: KGs grow continuously, and retraining from scratch is impractical.

Table 12.1: Inductive KG reasoning methods

Method	Approach	Inductive on
GraIL (Teru et al., 2020)	Subgraph GNN for rule extraction	Entities
NBFNet (Zhu et al., 2021)	Label propagation GNN	Entities
ULTRA (Galkin et al., 2024)	Pre-trained on relation structure	Both entities and relations
KG-ICL (Zhu et al., 2025)	In-context learning over KG	Full graph

Definition 12.2 (ULTRA (Galkin et al., 2024))**.** ULTRA pre-trains a model on diverse KGs. At inference, given a *new* KG:

1. Construct a relation graph $G_{\mathcal{R}}$ where nodes are relations and edges indicate relation co-occurrence.
2. Run a GNN on $G_{\mathcal{R}}$ to compute relation representations.
3. Use these representations as the message functions for NBFNet on the target KG.

Key insight: relation graph structure generalises across KGs even when entities and relations are entirely different.

12.4 Neuro-Symbolic AI

The convergence of neural learning and symbolic reasoning:

Differentiable rule learning: Learn logical rules via continuous optimisation (NeuralLP, DRUM). Extract symbolic rules after training.

Constraint-guided learning: Use KG ontology constraints (type, range) to guide or prune neural predictions (DCA-Trie philosophy).

Abductive reasoning: Given observations O and a KG $\mathcal{K}$, infer the best explanation E such that $\mathcal{K} \cup E \models O$.

Probabilistic soft logic (PSL): Combine first-order logic rules with probabilistic weights, learned from data.

Example 12.3 (DCA-Trie as neuro-symbolic system)**.** DCA-Trie bridges neural and symbolic in two ways:

1. **Symbolic:** KG ontology provides type/range constraints as symbolic rules.
2. **Neural:** LLM scores remaining paths to select the answer.

The symbolic oracle prunes the search space; the neural model selects among survivors.

12.5 Quality Assurance

Completeness estimation: How much of the real-world knowledge is captured? Use rule mining (AMIE) to find patterns; if a rule predicts many missing triples, the KG is likely incomplete in that area.

Error detection: Learn a scoring model; triples with anomalously low scores are candidates for manual review.

Freshness tracking: Monitor entity update frequency; stale entities (no updates in $>T$ days) may need refreshing.

Fairness auditing: Check for representation bias: are certain demographic groups over- or under-represented in the KG?

12.6 Exercises

Exercise 12.1. Explain why temporal KG embeddings require additional parameters compared to static embeddings. Estimate the parameter increase for TComplEx vs. Complex with $|\mathcal{T}|$ timestamps.

Exercise 12.2. Implement a simple inductive link predictor using GraIL's subgraph extraction: given $(h, r, t) \in \mathcal{K}_{\text{train}}$, extract the k-hop enclosing subgraph and run a GNN to classify whether (h, r, t) holds.

Exercise 12.3. Discuss the trade-offs between materialised reasoning (forward chaining) and query-time reasoning (backward chaining) for a KG with 1B triples and frequent updates.

Chapter 13

Learning Roadmap

Overview. This chapter provides a structured 9-week self-study plan covering the full spectrum of knowledge graph topics — from graph theory fundamentals to cutting-edge research. Each week includes reading, practice exercises, and project work.

13.1 Phase 1: Foundations (Weeks 1–3)

Week 1: Graph theory and RDF basics.
- Read: Chapter 1 of this book + Diestel Chapters 1–3.
- Practice: Build a small RDF graph with rdflib in Python.
- Project: Model your personal music library as RDF triples using Turtle syntax.
- Milestone: Load, query, and manipulate an RDF graph of >100 triples.

Week 2: SPARQL querying.
- Read: Chapter 6, Sections 6.1 and 6.3.
- Practice: Query Wikidata via its public SPARQL endpoint (query.wikidata.org).
- Project: Write 10 SPARQL queries of increasing complexity (1-hop → 3-hop with FILTER).
- Milestone: Retrieve multi-hop paths from a public KG.

Week 3: Ontology design.
- Read: Chapter 4.
- Practice: Model a small domain (e.g., university) in Protege with OWL 2 axioms.
- Project: Add disjointness, cardinality restrictions, and a transitive property. Run the HermiT reasoner.
- Milestone: Design and validate an OWL ontology with 10+ classes and 5+ object properties.

13.2 Phase 2: Construction and Querying (Weeks 4–5)

Week 4: KG construction pipeline.
- Read: Chapter 5.
- Practice: Run spaCy NER on Wikipedia text. Extract entities and relations.
- Project: Build an end-to-end pipeline: text → NER → RE → entity linking → RDF store.
- Milestone: Construct a KG of 500+ triples from unstructured text.

Week 5: Neo4j and Cypher.
- Read: Chapter 6, Section 6.2.
- Practice: Install Neo4j (local or Docker). Load the Week 4 KG.
- Project: Write 10 Cypher queries: MATCH, WHERE, variable-length paths, shortestPath, aggregation.
- Milestone: Query a property graph database with Cypher.

13.3 Phase 3: ML Integration (Weeks 6–7)

Week 6: KG embeddings.
- Read: Chapter 7.
- Practice: Train TransE and RotatE on FB15k-237 using PyKEEN.
- Project: Evaluate link prediction performance. Compare MRR and Hits@10 across models.
- Milestone: Reproduce a published KG embedding result.

Week 7: Graph Neural Networks.
- Read: Chapter 8.
- Practice: Implement GCN and GAT layers in PyTorch Geometric.
- Project: Train a GNN for node classification on a citation network (Cora, PubMed).
- Milestone: Achieve >80% accuracy on Cora node classification.

13.4 Phase 4: LLM Integration and Research (Weeks 8–9)

Week 8: GraphRAG.
- Read: Chapter 9.
- Practice: Build a simple GraphRAG pipeline with Neo4j + LLM (OpenAI or local).
- Project: Compare standard RAG vs. GraphRAG on 10 multi-hop questions.
- Milestone: GraphRAG pipeline answering multi-hop questions correctly.

Week 9: Constrained decoding and KGQA.
- Read: Chapter 10.
- Practice: Implement a GCR-style trie for path-constrained decoding.
- Project: Run the DCA-Trie codebase on WebQSP. Compare Hits@1 with the GCR baseline.
- Milestone: Reproduce a constrained decoding result and analyse the pruning statistics.

13.5 Weekly Commitment

Table 13.1: Estimated weekly time commitment

Activity	Hours
Reading	3–4
Practice exercises	3–4
Project work	4–6

Total: 10–14 hours/week. Learners with stronger backgrounds may complete Phases 1–2 in 2 weeks and dedicate more time to Phases 3–4.

13.6 Advanced Extensions

After completing the 9-week plan:

1. **Research paper reproduction:** Pick a paper from Chapter 12 and reproduce its results.

2. **Novel contribution:** Identify a limitation in DCA-Trie (e.g., the type oracle could be extended with OWL reasoning) and implement an improvement.

3. **Production deployment:** Deploy a GraphRAG system with real data (company documents, customer support tickets).

Chapter 14

Resource Index

Overview. This chapter provides a comprehensive, annotated index of resources for further study. Resources are organised by category and annotated with difficulty level, relevance, and accessibility.

14.1 Books

Table 14.1: Recommended books

Title	Author(s)	Link	L
Knowledge Graphs	Hogan et al.	https://kg-book.com/	Int
Graph Theory	Diestel	https://diestel-graph-theory.com/	A
Graph Representation Learning	Hamilton	https://www.cs.mcgill.ca/~wlh/grl_book/	A
Networks, Crowds, and Markets	Easley & Kleinberg	https://www.cs.cornell.edu/home/kleinber/networks-book/	I
Foundations of Data Science	Blum, Hopcroft, Kannan	https://www.cs.cornell.edu/jeh/book.pdf	Int
Semantic Web for the Working Ontologist	Allemang, Hendler, Gandon	Morgan Kaufmann	Int

14.2 Courses

1. **Knowledge Graphs (Krötzsch), TU Dresden.** Full lecture series with videos.
   https://iccl.inf.tu-dresden.de/web/Knowledge_Graphs/en
2. **CS224W: Machine Learning with Graphs, Stanford.** Comprehensive ML + graphs.
   https://web.stanford.edu/class/cs224w/
3. **Graph Data Modeling (Neo4j Graph Academy).** Free, self-paced.
   https://graphacademy.neo4j.com/courses/modeling-fundamentals/
4. **Agentic Knowledge Graph Construction (DeepLearning.AI).** LLM-based KG building.
   https://www.deeplearning.ai/courses/agentic-knowledge-graph-construction/

5. **Knowledge Graphs for AI Agent API Discovery (DeepLearning.AI).** KG for agents.
   https://learn.deeplearning.ai/courses/knowledge-graphs-for-ai-agent-api-discovery/
6. **KDD 2026 KG Tutorial (Whang et al.).** Embeddings and GraphRAG.
   https://bdi-lab.github.io/kkg_kdd2026/
7. **SPARQL by Example (Cambridge Semantics).** Free interactive tutorial.
   https://www.cambridgesemantics.com/blog/semantic-university/learn-sparql/

14.3 Software and Tools

Table 14.2: Software tools

Tool	Purpose	Link
Neo4j	Leading graph database	https://neo4j.com/
Protege	Ontology editor (free)	https://protege.stanford.edu/
rdflib	Python RDF manipulation	https://rdflib.readthedocs.io/
PyKEEN	KG embeddings (Python)	https://github.com/pykeen/pykeen
PyTorch Geometric	GNNs (Python)	https://pytorch-geometric.readthedocs.io/
DGL	Deep Graph Library	https://www.dgl.ai/
Microsoft GraphRAG	LLM + KG pipeline	https://github.com/microsoft/graphrag
LightRAG	Lightweight GraphRAG	https://github.com/HKUDS/LightRAG
OpenKE	KG embeddings (THU)	https://github.com/thunlp/OpenKE
SPARQL wrapper	SPARQL from Python	https://rdflib.readthedocs.io/
HermiT	OWL reasoner	http://www.hermit-reasoner.com/
ELK	OWL 2 EL reasoner	https://github.com/liveontologies/elk-reasoner

14.4 Datasets

Table 14.3: Public KG datasets

Dataset	Size	Domain	Access
Wikidata	∼1.5B statements	General	https://query.wikidata.org/

Table 14.3: Public KG datasets (continued)

Dataset	Size	Domain	Access
DBpedia	∼40M entities	Wikipedia	https://www.dbpedia.org/
YAGO	∼10M entities	General	https://yago-knowledge.org/
ConceptNet	∼8M nodes	Commonsense	https://conceptnet.io/
Freebase (deprecated)	∼50M entities	General	https://developers.google.com/freebase
FB15k-237	15K entities, 237 relations	KGQA benchmark	https://www.microsoft.com/en-us/download/details.aspx?id=52312
WN18RR	41K entities, 11 relations	KGQA benchmark	https://github.com/TimDettmers/ConvE
WebQSP	4,737 questions	KGQA	https://github.com/kelvinguu/webqsp
ComplexWebQuestions	34,689 questions	KGQA	https://github.com/IBM/ComplexWebQuestions

14.5 Conferences and Venues

ICML / NeurIPS / ICLR: Primary ML venues; KG embedding and GNN papers.

ACL / EMNLP / NAACL: NLP + KG integration; KGQA, GraphRAG.

ISWC (International Semantic Web Conference): Core KG venue; ontologies, SPARQL, RDF.

WWW / KDD: Applied KG papers; construction, search, industrial systems.

AKBC (Automated Knowledge Base Construction): Focused on KG construction.

14.6 Reproduction Checklist

When reproducing a KG paper:

1. Ensure you have the exact dataset split used in the paper.
2. Check the evaluation protocol: filtered or unfiltered? Raw or standardised?
3. Verify the hyperparameters (embedding dimension d, learning rate, batch size, number of negative samples).
4. Run 5 random seeds and report mean ± std.
5. Compare against published numbers from the *same* test set.

Appendix A

Appendix: Mathematical Notation

Table A.1: Summary of mathematical notation

Symbol	Meaning	First used
$G = (V, E)$	Graph with vertices V and edges E	§1.1
$\mathcal{K} = (\mathcal{E}, \mathcal{R}, T)$	Knowledge graph with entities, relations, triples	§2.1
$A \in \{0,1\}^{n \times n}$	Adjacency matrix	§1.2
$L = D - A$	Graph Laplacian	§1.4
$\mathbb{R}^d$	d-dimensional real vector space	§7.1
$\mathbb{C}^d$	d-dimensional complex vector space	§7.2
$\mathbb{1}$	All-ones vector	§1.4
$\|\cdot\|_p$	L_p norm	§7.2
$\circ$	Hadamard (element-wise) product	§7.2

Bibliography

[1] A. Bordes, N. Usunier, A. Garcia-Durán, J. Weston, and O. Yakhnenko. Translating embeddings for modeling multi-relational data. In *NeurIPS*, 2013.

[2] N. De Cao, G. Izs, L. Kovács, and I. Titov. Autoregressive entity retrieval. In *ICLR*, 2021.

[3] D. Easley and J. Kleinberg. *Networks, Crowds, and Markets*. Cambridge University Press, 2010.

[4] M. Galkin, X. Yuan, H. Ren, and J. Tang. Towards foundation models for knowledge graph reasoning. In *ICLR*, 2024.

[5] W. Hamilton, Z. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *NeurIPS*, 2017.

[6] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Mel, C. Gutierrez, S. Kirrane, J. Gayo, R. Navigli, S. Neumaier, A. Ngomo, A. Polleres, S. Rashid, A. Rula, L. Schmelzeisen, J. Seco, M. van Erp, and M. Zimmermann. *Knowledge Graphs*. Morgan & Claypool, 2021.

[7] T. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.

[8] T. Li, S. Zhang, and D. Zhou. Dynamic oracle-guided decoding for knowledge graph question answering. In *ACL*, 2025.

[9] R. Luo, A. Garcia-Durán, and Y. Zhang. Graph-constrained reasoning: Faithful multi-hop question answering with language models. In *ICML*, 2025.

[10] C. Mavromatis and G. Karypis. GNN-RAG: Graph neural retrieval for large language model reasoning. In *ACL Findings*, 2025.

[11] Z. Sun, Z. Deng, J. Nie, and J. Tang. Rotate: Knowledge graph embedding by relational rotation in complex space. In *ICLR*, 2019.

[12] T. Trouillon, J. Welbl, S. Riedel, E. Gaussier, and G. Bouchard. Complex embeddings for simple link prediction. In *ICML*, 2016.

[13] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. In *ICLR*, 2018.

[14] K. Xu, W. Hu, J. Leskovec, and S. Jegelka. How powerful are graph neural networks? In *ICLR*, 2019.

[15] B. Yang, W. Yih, X. He, J. Gao, and L. Deng. Embedding entities and relations for learning and inference in knowledge bases. In *ICLR*, 2015.

[16] M. Yasunaga, H. Ren, A. Bosselut, P. Liang, and J. Leskovec. QA-GNN: Reasoning with language models and knowledge graphs for question answering. In *NAACL*, 2021.

[17] Z. Zhu, Z. Zhang, L. Huang, and Y. He. Neural bellman-ford networks: A general graph neural network framework for link prediction. In *NeurIPS*, 2021.